**SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**

**Department of Artificial Intelligence and Data Science**

**ASSESSMENT TOOL 3 – ERROR ANALYSIS**

**SAMPLE SOLUTION (REFERENCE FOR STUDENTS)**

| **Course Code** | **DSA03** | **Course Title** | **Natural Language Processing** |
|---|---|---|---|
| CO Assessed | CO2 | Assessment | Assessment Tool 3 – Error Analysis |
| Weightage | 30% of CIA | Duration | 30 minutes |

Name: M S GOWRISH SHANKAR REDDY

Reg No:192424270

## Annexure A – Error Analysis

### Note on the experiments

The public datasets named in the questions (PubMed 20k RCT, Sentiment140, AG News or BBC News, Amazon reviews, 20 Newsgroups) were not available when this reference was prepared. Every program therefore runs on a small sample written for this solution. The line that loads the real dataset is shown as a comment, and students should replace the sample with it. The numbers shown describe the sample only and will differ on the full datasets.

Stemming uses the original Porter algorithm through the same interface as nltk.stem.PorterStemmer. A few words may stem differently in NLTK, because NLTK adds some extensions to the original algorithm. Where WordNet is mentioned, a small built-in word list stands in for it, and the comment in the code shows the NLTK call to use instead.

Instruction: answer any four questions (two scenario-based and two programming-based). All seven are solved here as a reference.

## Scenario-Based Error Analysis (1 – 4)

### Question 1 – Morphological Analysis Error in Biomedical Dataset

Problem: a clinical search engine stems every word with the Porter stemmer before morphological analysis. Searches for infection, infectious, infected and infect return inconsistent results.

### Task 1 – Preprocessing

- Convert the text to lowercase.
- Remove punctuation and split the text into tokens.
- Remove stop words such as the, of, in, and, were.

**Task 2 – Apply the Porter stemmer and record the stems**

The program below prints the stem of every word. The key results are infection → infect, infected → infect, infections → infect, infect → infect, but infectious → infecti.

```
# Q1: Why direct Porter stemming harms retrieval on biomedical text
import re, textwrap
from collections import defaultdict
from nltk.stem import PorterStemmer

ps = PorterStemmer()

# Sample clinical sentences (replace with the PubMed 20k RCT text for the full experiment)
reports = [
        "The infection spread quickly. Infected patients developed infectious symptoms.",
        "Doctors infect no one when infectious agents are contained; infections declined.",
        "General treatment of the disease needs careful generation of new antibiotics.",
        "The study of operations in generating vaccines was operative in the general hospital.",
]
STOP = {"the", "of", "in", "was", "are", "no", "one", "when", "new", "needs", "and", "were"}

# Step 1: preprocessing (lowercase, remove punctuation, remove stop words)
tokens = [w for r in reports for w in re.findall(r"[a-z]+", r.lower()) if w not in STOP]
vocab = sorted(set(tokens))

# Step 2: apply the Porter stemmer
stems = {w: ps.stem(w) for w in vocab}
print("PORTER STEMS")
print(textwrap.fill(", ".join(f"{w} -> {s}" for w, s in stems.items()), 88))

# Step 3: find stemming errors (ROOT = linguistic root of each word family)
ROOT = {"infect": "infect", "infection": "infect", "infections": "infect",
                "infected": "infect", "infectious": "infect",
                "general": "general", "generation": "generate", "generating": "generate",
                "operations": "operate", "operative": "operate"}

by_root = defaultdict(set)
by_stem = defaultdict(set)
for w, r in ROOT.items():
        by_root[r].add(stems[w])
        by_stem[stems[w]].add(r)

print("\nERROR TYPE A - related words split into different stems")
for r, s in by_root.items():
        if len(s) > 1:
                print(f"  root '{r}' -> stems {sorted(s)}")

print("ERROR TYPE B - unrelated words merged into one stem")
for s, r in by_stem.items():
        if len(r) > 1:
```

```
            words = [w for w in ROOT if stems[w] == s]
            print(f"  stem '{s}' <- {words} (roots {sorted(r)})")

# Step 4: treat inflection and derivation differently
BASE = {"infect", "infection", "infectious", "general", "generation", "generate",
            "operation", "operative", "operate", "patient", "symptom"}

def inflectional_lemma(word):
    """Remove only inflectional endings (-s, -es, -ed, -ing); keep derivational ones."""
    if word in BASE:
        return word
    for suffix in ("ing", "ed", "es", "s"):
        if word.endswith(suffix):
            stem = word[:-len(suffix)]
            for cand in (stem, stem + "e"):
                if cand in BASE:
                    return cand
    return word

print("\nINFLECTION-ONLY NORMALIZATION (corrected)")
for w in ["infect", "infection", "infections", "infected", "infectious",
              "general", "generation", "generating", "operations", "operative"]:
    print(f"  {w:<12} Porter: {stems[w]:<8} Inflectional lemma: {inflectional_lemma(w):<11}
    Root: {ROOT[w]}")
```

```
PORTER STEMS

agents -> agent, antibiotics -> antibiot, careful -> care, contained -> contain,

declined -> declin, developed -> develop, disease -> diseas, doctors -> doctor, general

-> gener, generating -> gener, generation -> gener, hospital -> hospit, infect ->

infect, infected -> infect, infection -> infect, infections -> infect, infectious ->

infecti, operations -> oper, operative -> oper, patients -> patient, quickly -> quickli,

spread -> spread, study -> studi, symptoms -> symptom, treatment -> treatment, vaccines

-> vaccin


ERROR TYPE A - related words split into different stems

root 'infect' -> stems ['infect', 'infecti']


ERROR TYPE B - unrelated words merged into one stem

stem 'gener' <- ['general', 'generation', 'generating'] (roots ['general', 'generate'])


INFLECTION-ONLY NORMALIZATION
```

## Output

PORTER STEMS
agents -> agent, antibiotics -> antibiot, careful -> care, contained -> contain,
declined -> declin, developed -> develop, disease -> diseas, doctors -> doctor, general
-> gener, generating -> gener, generation -> gener, hospital -> hospit, infect ->
infect, infected -> infect, infection -> infect, infections -> infect, infectious ->

```
infecti, operations -> oper, operative -> oper, patients -> patient, quickly -> quickli,
spread -> spread, study -> studi, symptoms -> symptom, treatment -> treatment, vaccines
-> vaccin

ERROR TYPE A - related words split into different stems
   root 'infect' -> stems ['infect', 'infecti']
ERROR TYPE B - unrelated words merged into one stem
   stem 'gener' <- ['general', 'generation', 'generating'] (roots ['general', 'generate'])

INFLECTION-ONLY NORMALIZATION (corrected)
   infect Porter: infect Inflectional lemma: infect Root: infect
   infection Porter: infect Inflectional lemma: infection Root: infect
   infections Porter: infect Inflectional lemma: infection Root: infect
   infected Porter: infect Inflectional lemma: infect Root: infect
   infectious Porter: infecti Inflectional lemma: infectious Root: infect
   general Porter: gener Inflectional lemma: general Root: general
   generation Porter: gener Inflectional lemma: generation Root: generate
   generating Porter: gener Inflectional lemma: generate Root: generate
   operations Porter: oper Inflectional lemma: operation Root: operate
   operative Porter: oper Inflectional lemma: operative Root: operate
```

**Task 3 – Words where stemming damages derivational information**

- infectious → infecti: the family of infect is split. The searcher who types infect does not match infectious, and infecti is not a real word.
- general, generation, generating → gener: unrelated words (general is an adjective, generate is a verb) are merged into one stem, so a search for one retrieves documents about the other.
- operations, operative → oper: a non-word stem that also loses the difference between a noun (operation) and an adjective (operative).

**Task 4 – Why inflectional and derivational morphology must be treated differently**

- Inflectional morphology (-s, -ed, -ing) changes only the grammatical form. The word keeps its meaning and word class, so infected and infect can safely share one index term.
- Derivational morphology (-ion, -ious, -ive) builds a new word with a new meaning or word class. In clinical text, infection (a condition) and infectious (a property of an agent) are not interchangeable.
- Porter removes both kinds with the same suffix rules and has no dictionary, so it produces non-words (infecti) and collisions (gener).
- Correction: remove only inflectional endings for the index, keep derived words as separate terms, and store the derivational link (infectious → infect) in a lexicon for optional query expansion. The program shows this in its last block.

**Conclusion:** Porter stemming should not be applied blindly to biomedical text. Inflection-only normalization keeps retrieval consistent without losing derivational meaning.

**Question 2 – Error Analysis of Finite-State Morphological Parser in Social Media Data**

Problem: a finite-state parser analyses customer reviews correctly for simple words but fails on happiest, unbelievable, running, reordering, smartphones and unreadable.

**Task 1 – Words with incorrect analysis**

All six listed words fail in the original parser. On the 14-word test list, 12 words fail (see the output).

**Task 2 – Why the finite-state parser fails**

- It allows only one affix: either one suffix or one prefix. Words such as unbelievable (un- + believe + -able) need both.
- It matches the stem exactly. It has no spelling-change transitions, so happi(est), runn(ing) and believ(able) are never mapped back to happy, run and believe.
- It cannot analyse compounds such as smartphones (smart + phone + -s).
- It cannot loop over several suffixes, as in players (play + -er + -s).

**Task 3 – Modified transitions**

- Start state: loop on prefix arcs (un-, re-, dis-), so zero or more prefixes are read.
- Root state: accept a lexicon root or a compound of two roots (smart + phone).
- Suffix state: loop on suffix arcs (-ness, -ment, -able, -ing, -est, -ed, -er, -s), so zero or more suffixes are read.
- Spelling arcs: restore a deleted e (believ → believe), an undoubled consonant (runn → run), or y after i (happi → happy, cit + ies → city).

```
# Q2: Finite-state style morphological parser - before and after correction

LEX = {"happy", "believe", "run", "order", "smart", "phone", "read", "play",
       "agree", "city", "walk", "car"}
PREFIXES = ["un", "re", "dis"]
SUFFIXES = {"ness": "ness", "ment": "ment", "able": "able", "ing": "ing", "est": "est",
            "ies": "s", "ed": "ed", "er": "er", "es": "s", "s": "s"}

def fmt(prefixes, root, suffixes):
    return " + ".join([p + "-" for p in prefixes] + [root] + ["-" + s for s in suffixes])

# ---------- BEFORE: one affix only, exact stem match, no spelling rules ----------
def naive_parse(word):
    if word in LEX:
        return word
    for suf, label in SUFFIXES.items():
        if word.endswith(suf) and word[:-len(suf)] in LEX:
            return fmt([], word[:-len(suf)], [label])
    for p in PREFIXES:
        if word.startswith(p) and word[len(p):] in LEX:
            return fmt([p], word[len(p):], [])
    return "FAIL"
```

```
# ---------- AFTER: prefix* root suffix* with spelling-change transitions ----------
def candidates(base, suf):
    out = [base, base + "e"]                        # e-deletion: believ(e)able
    if len(base) > 2 and base[-1] == base[-2]:
        out.append(base[:-1])                       # consonant doubling: runn(ing)
    if base.endswith("i"):
        out.append(base[:-1] + "y")                 # y -> i: happi(est)
    if suf == "ies":
        out.append(base + "y")                      # cit(ies) -> city
    return out

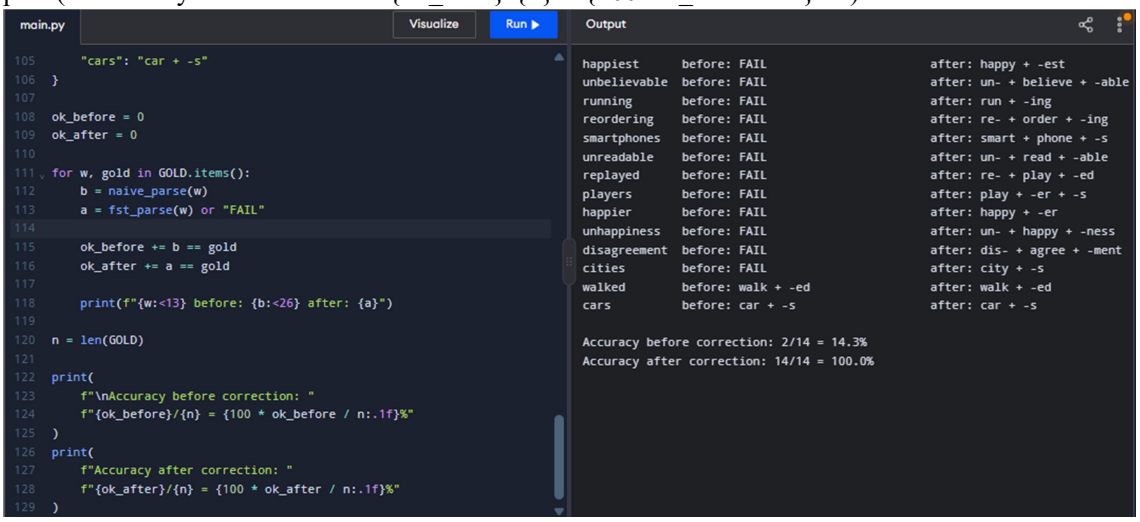
def root_with_prefixes(w):
    if w in LEX:
        return [], w
    for i in range(2, len(w) - 1):                  # compound: smart + phone
        if w[:i] in LEX and w[i:] in LEX:
            return [], w[:i] + " + " + w[i:]
    for p in PREFIXES:
        if w.startswith(p):
            r = root_with_prefixes(w[len(p):])
            if r:
                return [p] + r[0], r[1]
    return None

def fst_parse(word, suffixes=()):
    r = root_with_prefixes(word)
    if r:
        return fmt(r[0], r[1], suffixes)
    for suf in sorted(SUFFIXES, key=len, reverse=True):
        if word.endswith(suf):
            for c in candidates(word[:-len(suf)], suf):
                if len(c) > 1:
                    res = fst_parse(c, (SUFFIXES[suf],) + suffixes)
                    if res:
                        return res
    return None

GOLD = {
    "happiest": "happy + -est", "unbelievable": "un- + believe + -able",
    "running": "run + -ing", "reordering": "re- + order + -ing",
    "smartphones": "smart + phone + -s", "unreadable": "un- + read + -able",
    "replayed": "re- + play + -ed", "players": "play + -er + -s",
    "happier": "happy + -er", "unhappiness": "un- + happy + -ness",
    "disagreement": "dis- + agree + -ment", "cities": "city + -s",
    "walked": "walk + -ed", "cars": "car + -s",
}

ok_before = ok_after = 0
```

```
for w, gold in GOLD.items():
        b, a = naive_parse(w), fst_parse(w) or "FAIL"
        ok_before += b == gold
        ok_after += a == gold
        print(f"{w:<13} before: {b:<26} after: {a}")
n = len(GOLD)
print(f"\nAccuracy before correction: {ok_before}/{n} = {100*ok_before/n:.1f}%")
print(f"Accuracy after correction : {ok_after}/{n} = {100*ok_after/n:.1f}%")
```




### Output (Task 4 – accuracy before and after)

happiest before: FAIL after: happy + -est
unbelievable before: FAIL after: un- + believe + -able
running before: FAIL after: run + -ing
reordering before: FAIL after: re- + order + -ing
smartphones before: FAIL after: smart + phone + -s
unreadable before: FAIL after: un- + read + -able
replayed before: FAIL after: re- + play + -ed
players before: FAIL after: play + -er + -s
happier before: FAIL after: happy + -er
unhappiness before: FAIL after: un- + happy + -ness
disagreement before: FAIL after: dis- + agree + -ment
cities before: FAIL after: city + -s
walked before: walk + -ed after: walk + -ed
cars before: car + -s after: car + -s

Accuracy before correction: 2/14 = 14.3%
Accuracy after correction : 14/14 = 100.0%
The test list and lexicon are small and were built for this exercise, so 100% accuracy is not expected on real tweets. Unknown words, slang, hashtags and misspellings would still fail.

### Task 5 – Computational complexity of the modified FST

- The original parser makes one pass over the word and tries each affix once, so it takes O(n · A) time for a word of length n and A affixes.

- The modified parser tries up to c spelling candidates (about four) for each of up to d stacked suffixes, so the worst case is O((c · A)^d · n). Because d is at most three and A is small, it stays fast in practice.
- If the lexicon and the spelling rules are compiled into one transducer, analysis becomes O(n), but the number of states grows (roughly the product of the lexicon automaton and the rule automata), so memory use increases.
- The modified parser can return several analyses (unreadable = un- + readable or unread + -able). These must be ranked or filtered, which adds further cost.

**Question 3 – Error Analysis of Stemming in News Article Classification**

Problem: a news classifier stems words first. Technology articles are misclassified as business, and the words organization, organizer, organizing, organized and organization's seem to be mapped inconsistently.

```
# Q3: Stemming vs lemmatization for news classification
import re
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import cross_val_predict, LeaveOneOut
from sklearn.metrics import accuracy_score, confusion_matrix

ps = PorterStemmer()

# Small labelled sample (replace with AG News / BBC News for the full experiment)
TECH = [
    "the organization released a new software update for developers",
    "organizers of the technology conference announced new chip designs",
    "engineers organized the data into a faster database system",
    "organizing large datasets requires scalable computing and clever algorithms",
    "the organization's cloud platform runs machine learning models",
    "developers are organizing open source code repositories",
    "new processors improve computing speed for mobile devices",
    "researchers trained neural networks on massive image datasets",
    "the company released a secure operating system for servers",
    "programmers organized the software modules into libraries",
    "wireless networks connect devices using faster protocols",
    "the organizer of the hackathon demonstrated robotics and automation",
]
BUSINESS = [
    "the organization reported higher quarterly profits and revenue",
    "organizers of the trade summit discussed market regulations",
    "executives organized the merger of two large banks",
    "organizing corporate finances requires careful budgeting and auditing",
    "the organization's shareholders approved the annual dividend",
```

```
        "investors are organizing a new venture capital fund",
        "stock markets rose as inflation and interest rates fell",
        "retailers increased prices after higher costs hit profits",
        "the company announced layoffs to cut operating expenses",
        "managers organized the quarterly sales targets by region",
        "banks lend money to businesses at lower interest rates",
        "the organizer of the investment forum praised economic growth",
]
docs = TECH + BUSINESS
labels = [0] * len(TECH) + [1] * len(BUSINESS)       # 0 = technology, 1 = business

LEMMA = {"organizing": "organize", "organized": "organize", "organizers": "organizer",
                "developers": "developer", "designs": "design", "engineers": "engineer",
                "datasets": "dataset", "algorithms": "algorithm", "models": "model",
                "repositories": "repository", "processors": "processor", "devices": "device",
                "researchers": "researcher", "networks": "network", "servers": "server",
                "programmers": "programmer", "modules": "module", "libraries": "library",
                "protocols": "protocol", "profits": "profit", "regulations": "regulation",
                "executives": "executive", "finances": "finance", "shareholders": "shareholder",
                "investors": "investor", "markets": "market", "rates": "rate", "retailers": "retailer",
                "prices": "price", "costs": "cost", "layoffs": "layoff", "expenses": "expense",
                "managers": "manager", "targets": "target", "businesses": "business",
                "announced": "announce", "released": "release", "trained": "train",
                "increased": "increase", "approved": "approve", "rose": "rise", "fell": "fall",
                "discussed": "discuss", "reported": "report", "requires": "require", "runs": "run",
                "praised": "praise", "demonstrated": "demonstrate", "using": "use", "banks": "bank"}

def tokens(text):
        return re.findall(r"[a-z']+", text.lower())          # keeps organization's as one token

def none(text):  return tokens(text)
def porter(text): return [ps.stem(t) for t in tokens(text)]
def lemma(text):
        return [LEMMA.get(re.sub(r"'s$", "", t), re.sub(r"'s$", "", t)) for t in tokens(text)]

print("Task 2 - Porter stems of the problem words")
for w in ["organization", "organizer", "organizing", "organized", "organization's",
                "organ", "organic", "organism"]:
        print(f"  {w:<16} -> {ps.stem(w)}")

print("\nTask 4 - classifier comparison (Naive Bayes, leave-one-out)")
for name, f in [("Without stemming", none), ("Porter stemming", porter), ("Lemmatization", lemma)]:
        vec = CountVectorizer(analyzer=f)
        X = vec.fit_transform(docs)
        pred = cross_val_predict(MultinomialNB(), X, labels, cv=LeaveOneOut())
        cm = confusion_matrix(labels, pred)
        print(f"  {name:<17} vocabulary = {X.shape[1]:<3} accuracy = {accuracy_score(labels,
        pred):.3f}"
                f"  confusion matrix = {cm.tolist()}")
```

```
Running code
Task 2 - Porter stems of the problem words
organization     -> organ
organizer        -> organ
organizing       -> organ
organized        -> organ
organization's   -> organization'
organ            -> organ
organic          -> organ
organism         -> organ

Task 4 - Classifier comparison
Without stemming  vocabulary = 132 accuracy = 0.500 confusion matrix = [[7, 5], [7, 5]]
Porter stemming   vocabulary = 127 accuracy = 0.583 confusion matrix = [[7, 5], [5, 7]]
Lemmatization     vocabulary = 128 accuracy = 0.583 confusion matrix = [[7, 5], [5, 7]]
Run completed in 5,174.500ms
```

## Output

Task 2 - Porter stems of the problem words
organization -> organ
organizer -> organ
organizing -> organ
organized -> organ
organization's -> organization'
organ -> organ
organic -> organ
organism -> organ

Task 4 - classifier comparison (Naive Bayes, leave-one-out)
Without stemming vocabulary = 132 accuracy = 0.500 confusion matrix = [[7, 5], [7, 5]]
Porter stemming vocabulary = 127 accuracy = 0.583 confusion matrix = [[7, 5], [5, 7]]
Lemmatization vocabulary = 128 accuracy = 0.583 confusion matrix = [[7, 5], [5, 7]]

### Task 1 – How stemming affects feature extraction

- Stemming merges word forms, so the vocabulary becomes smaller (132 features became 127 in the sample) and counts of related forms are added together.
- The merging is only useful when the merged words really share one meaning. When different words are merged, the feature loses its meaning and can no longer tell the classes apart.

### Task 2 – Incorrect stems generated by the Porter stemmer

- organization, organizer, organizing and organized all become organ. They are consistent with each other, but they are now identical to organ, organic and organism, which belong to other topics (biology, food).

- organization's becomes organization' (with an apostrophe). The possessive is not removed, so this form becomes a separate feature. This is the real inconsistency, and it comes from tokenization, not from the stemmer.

**Task 3 – Would lemmatization improve classification?**

Yes, in principle. A lemmatizer maps organizing and organized to organize, removes the possessive from organization's, and keeps organizer and organization separate from organ. It also uses a dictionary, so it does not create non-words.

**Task 4 – Classifier comparison**

The sample has only 24 documents, so a difference of one or two documents moves the accuracy by several points. Accuracy was 0.500 without stemming, 0.583 with Porter stemming and 0.583 with lemmatization. These values do not allow a ranking of the methods. The same program should be run on AG News or BBC News for a reliable comparison.

**Task 5 – Strategy that minimizes semantic information loss**

Lemmatization, with possessives removed during tokenization, loses the least meaning, because it only removes inflection and checks the result against a dictionary. Porter stemming is acceptable only when vocabulary size or speed matters more than precision.

**Conclusion:** The misclassification is caused by over-stemming (many different words reduced to organ) and by the untreated possessive form, not by randomly inconsistent stemming.

**Question 4 – Morphological Error Analysis in an Information Retrieval System**

Problem: an e-commerce search engine uses the Porter stemmer. Queries such as watches, watching, washable, washer and washed give less relevant results.

```
# Q4: Stemming errors in an e-commerce product search
import re
from nltk.stem import PorterStemmer

ps = PorterStemmer()

# Task 1 and 2: Porter output for the complaint words and error category
CASES = [  # word, expected form, error category
        ("watches",  "watch",    "Inflectional (correct stem, but noun and verb senses get mixed)"),
        ("watching", "watch",    "Inflectional (correct stem, but verb sense mixed with noun)"),
        ("washed",   "wash",     "Inflectional (correct)"),
        ("washable", "washable", "Derivational (-able cut into a non-word)"),
        ("washer",   "washer",   "Derivational (-er kept, link to wash lost)"),
]
print("PORTER OUTPUT")
for w, exp, cat in CASES:
        s = ps.stem(w)
        print(f"  {w:<9} -> {s:<9} expected: {exp:<9} {cat}")

# Task 4: improved pipeline = POS-aware inflectional lemmatization only
```

```
TAGGED = {"watches": ("watch", "N"), "watch": ("watch", "N"), "watching": ("watch", "V"),
          "washed": ("wash", "V"), "washing": ("wash", "V"), "washable": ("washable", "ADJ"),
          "washer": ("washer", "N")}

CATALOG = {
    1: "Men's analog wrist watches with leather strap",
    2: "Smart watch fitness tracker with heart rate monitor",
    3: "Washable cotton bed sheets soft and breathable",
    4: "Front load washer and dryer combo for home laundry",
    5: "Portable mini washer for travel and dorm rooms",
    6: "Projector for watching movies outdoors",
    7: "Denim jeans pre washed for a soft finish",
    8: "Hand washing liquid soap pack of three",
}
GOLD = {"watches": {1, 2}, "watching": {6}, "washable": {3}, "washer": {4, 5}}

def porter_terms(text):
    return {ps.stem(t) for t in re.findall(r"[a-z]+", text.lower())}

def improved_terms(text):
    terms = set()
    for t in re.findall(r"[a-z]+", text.lower()):
        lemma, pos = TAGGED.get(t, (t, "X"))
        terms.add(f"{lemma}/{pos}")
    return terms

def search(query, index, analyzer):
    q = analyzer(query)
    return {d for d, terms in index.items() if q & terms}

porter_idx   = {d: porter_terms(t) for d, t in CATALOG.items()}
improved_idx = {d: improved_terms(t) for d, t in CATALOG.items()}

def prf(found, gold):
    p = len(found & gold) / len(found) if found else 0
    r = len(found & gold) / len(gold)
    return p, r

print("\nRETRIEVAL (relevant documents in brackets)")
print(f"  {'query':<9} {'gold':<8} {'Porter result':<14} P/R         {'Improved result':<16} P/R")
for q, gold in GOLD.items():
    a = search(q, porter_idx, porter_terms)
    b = search(q, improved_idx, improved_terms)
    pa, ra = prf(a, gold); pb, rb = prf(b, gold)
    print(f"  {q:<9} {str(sorted(gold)):<8} {str(sorted(a)):<14} {pa:.2f}/{ra:.2f}
    {str(sorted(b)):<16} {pb:.2f}/{rb:.2f}")
```

```
Installing packages
Running code
PORTER OUTPUT
watches   -> watch     expected: watch     Inflectional (correct stem, but noun and verb senses get mixed)
watching  -> watch     expected: watch     Inflectional (correct stem, but verb sense mixed with noun)
washed    -> wash      expected: wash      Inflectional (correct)
washable  -> washabl   expected: washable  Derivational (-able cut into a non-word)
washer    -> washer    expected: washer    Derivational (-er kept, link to wash lost)

RETRIEVAL
query     gold     Porter result  P/R Improved result  P/R
watches   [1, 2]   [1, 2, 6]      0.67/1.00 [1, 2]          1.00/1.00
watching  [6]      [1, 2, 6]      0.33/1.00 [6]             1.00/1.00
washable  [3]      [3]            1.00/1.00 [3]             1.00/1.00
washer    [4, 5]   [4, 5]         1.00/1.00 [4, 5]          1.00/1.00
Run completed in 2,617.500ms
```

### Output

PORTER OUTPUT

watches -> watch expected: watch Inflectional (correct stem, but noun and verb senses get mixed)
watching -> watch expected: watch Inflectional (correct stem, but verb sense mixed with noun)
washed -> wash expected: wash Inflectional (correct)
washable -> washabl expected: washable Derivational (-able cut into a non-word)
washer -> washer expected: washer Derivational (-er kept, link to wash lost)

Retrieval results on the sample catalogue (P = precision, R = recall):

| Query | Relevant | Porter result | Porter P / R | Improved result | Improved P / R |
|---|---|---|---|---|---|
| watches | [1, 2] | [1, 2, 6] | 0.67/1.00 | [1, 2] | 1.00/1.00 |
| watching | [6] | [1, 2, 6] | 0.33/1.00 | [6] | 1.00/1.00 |
| washable | [3] | [3] | 1.00/1.00 | [3] | 1.00/1.00 |
| washer | [4, 5] | [4, 5] | 1.00/1.00 | [4, 5] | 1.00/1.00 |

### Task 1 – Stemming errors that affect relevance

- watches and watching are both reduced to watch. A search for the product watches also returns documents about watching movies (precision 0.67), and a search for watching returns the wrist watches (precision 0.33).
- washable becomes washabl, which is not a word. The suffix is cut halfway (the final e is removed in the last Porter step), so the term cannot be linked to wash or shown to the user.
- washer stays washer, so the link to wash is lost, although washer is also a different product from the verb wash.

**Task 2 – Inflectional or derivational**

- Inflectional: watches → watch, watching → watch, washed → wash. The stems are correct, but the noun and the verb sense are mixed.
- Derivational: washable → washabl (-able) and washer → washer (-er).

**Task 3 – Why some suffixes should not be removed**

Suffixes such as -able, -er and -ness create new words with a new word class or meaning. Washable is an adjective (can be washed), washer is a machine or ring. Removing them changes what the shopper is asking for. Stripping them also produces stems that are not words, which cannot be shown in suggestions or matched to product attributes.

**Task 4 – Improved morphological preprocessing strategy**

1. Tokenize and lowercase the product titles and the queries.
2. Tag each token with its part of speech (noun, verb, adjective).
3. Lemmatize inflection only, using the tag: watches (noun) and watching (verb) get different index terms.
4. Keep derived words (washable, washer) as separate terms and store their relation to the root (washable → wash) for optional query expansion with a lower weight.
5. Keep an exception list of product terms that must never be changed.

## Programming-Based Error Analysis (5 – 7)

**Question 5 – Identify the Error, Rectify It, and Analyze the Output**

**Task 1 – Coding errors**

- data["Text"].apply(ps.stem) passes a whole document to stem(), which expects one word. The stemmer treats the entire row as a single word.
- There is no tokenization, so the text is never split into words.
- There is no punctuation removal or lowercase handling before stemming.
- The column name and file name are assumed. The commonly used BBC file (bbc-text.csv) has the lowercase column name text, so "Text" can raise a KeyError.

**Task 2 – Why the output is incorrect**

Only the end of each document is changed (the last word loses its suffix), because the stemmer sees one long string. Everything else stays as it was, apart from lowercase conversion. In the first row below, only problems became problem.

**Task 3 and 4 – Corrected program and its execution**

```
# Q5: Porter stemming on a news dataset - original (faulty) code vs corrected code
import re
import pandas as pd
from nltk.stem import PorterStemmer
```

```
ps = PorterStemmer()

# data = pd.read_csv("bbc-text.csv")      # real BBC file; columns are "category" and "text"
data = pd.DataFrame({"text": [            # small sample used to run this reference solution
        "The government announced new policies as companies and cities reported various economic
        problems",
        "News about the university studies showed that parties in many countries tried to tell stories
        about electricity",
        "Communication between communities is generally reliable and local organization carried the
        main work",
        "The business was operating happily while a new generation was studying economic reports",
        "Officials said the responsible leaders had visited the stories",
]})

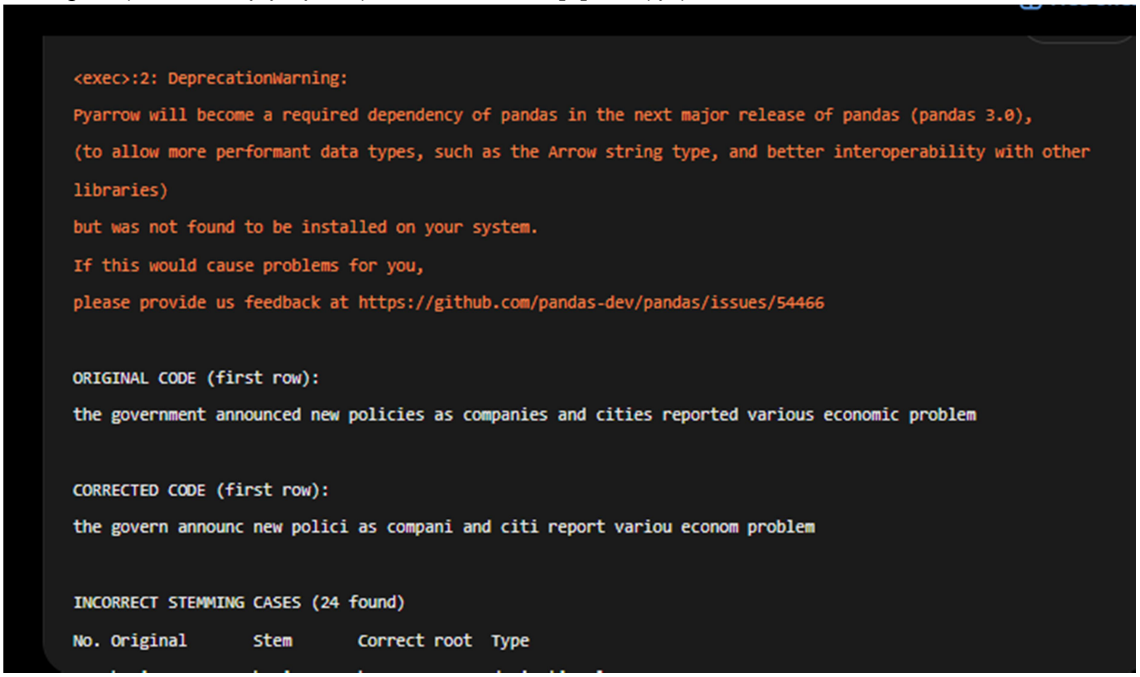
# ---- ORIGINAL (faulty) code ----
data["Original_Result"] = data["text"].apply(ps.stem)      # stems the WHOLE sentence as one word
print("ORIGINAL CODE (first row):")
print(" ", data["Original_Result"][0])

# ---- CORRECTED code ----
def stem_text(text):
        tokens = re.findall(r"[a-z]+", text.lower())           # tokenize + lowercase
        return " ".join(ps.stem(t) for t in tokens)            # stem every word separately

data["Processed"] = data["text"].apply(stem_text)
print("\nCORRECTED CODE (first row):")
print(" ", data["Processed"][0])

# ---- Task 5/6: original words vs stems, flagging incorrect stems ----
# word: (correct root, morphology type)
GOLD = {
        "policies": ("policy", "inflectional"), "companies": ("company", "inflectional"),
        "cities": ("city", "inflectional"), "countries": ("country", "inflectional"),
        "parties": ("party", "inflectional"), "stories": ("story", "inflectional"),
        "studies": ("study", "inflectional"), "studying": ("study", "inflectional"),
        "carried": ("carry", "inflectional"), "tried": ("try", "inflectional"),
        "operating": ("operate", "inflectional"), "communities": ("community", "inflectional"),
        "news": ("news", "inflectional rule misapplied"), "various": ("various", "inflectional rule
        misapplied"),
        "happily": ("happy", "derivational"), "economic": ("economy", "derivational"),
        "generally": ("general", "derivational"), "generation": ("generate", "derivational"),
        "organization": ("organize", "derivational"), "electricity": ("electric", "derivational"),
        "communication": ("communicate", "derivational"), "reliable": ("rely", "derivational"),
        "responsible": ("respond", "derivational"), "business": ("busy", "derivational"),
        "university": ("universe", "derivational"),
}
words = sorted({w for t in data["text"] for w in re.findall(r"[a-z]+", t.lower())})
bad = [(w, ps.stem(w), *GOLD[w]) for w in words if w in GOLD and ps.stem(w) != GOLD[w][0]]
```

```
print(f"\nINCORRECT STEMMING CASES ({len(bad)} found)")
print(f"{'No.':<4}{'Original':<15}{'Stem':<11}{'Correct root':<14}Type")
for i, (w, s, root, kind) in enumerate(bad, 1):
    print(f"{i:<4}{w:<15}{s:<11}{root:<14}{kind}")
for k in ("inflectional", "inflectional rule misapplied", "derivational"):
    print(f"Count - {k}: {sum(1 for b in bad if b[3] == k)}")
```




## Output

ORIGINAL CODE (first row):

the government announced new policies as companies and cities reported various economic problem

CORRECTED CODE (first row):

the govern announc new polici as compani and citi report variou econom problem

## Task 5 and 6 – Original words and stems: incorrect cases

INCORRECT STEMMING CASES (25 found)

| No. | Original word | Stem | Correct root | Morphology type |
|---|---|---|---|---|
| 1 | business | busi | busy | derivational |
| 2 | carried | carri | carry | inflectional |
| 3 | cities | citi | city | inflectional |
| 4 | communication | commun | communicate | derivational |
| 5 | communities | commun | community | inflectional |
| 6 | companies | compani | company | inflectional |
| 7 | countries | countri | country | inflectional |

| No. | Original word | Stem | Correct root | Morphology type |
|---|---|---|---|---|
| 8 | economic | econom | economy | derivational |
| 9 | electricity | electr | electric | derivational |
| 10 | generally | gener | general | derivational |
| 11 | generation | gener | generate | derivational |
| 12 | happily | happili | happy | derivational |
| 13 | news | new | news | inflectional rule misapplied |
| 14 | operating | oper | operate | inflectional |
| 15 | organization | organ | organize | derivational |
| 16 | parties | parti | party | inflectional |
| 17 | policies | polici | policy | inflectional |
| 18 | reliable | reliabl | rely | derivational |
| 19 | responsible | respons | respond | derivational |
| 20 | stories | stori | story | inflectional |
| 21 | studies | studi | study | inflectional |
| 22 | studying | studi | study | inflectional |
| 23 | tried | tri | try | inflectional |
| 24 | university | univers | universe | derivational |
| 25 | various | variou | various | inflectional rule misapplied |

- inflectional: 12
- inflectional rule misapplied: 2
- derivational: 11

**Analysis of the incorrect cases**

- Inflectional errors (policies, companies, cities, studies, tried): Porter turns y or ies into i, so the stems (polici, compani, citi, studi, tri) are not words and do not match the singular form policy, company, city, study, try.
- Misapplied inflection rule (news → new, various → variou): the final s is removed although it is not a plural ending.
- Derivational errors (organization → organ, generation → gener, communication → commun, university → univers): too many suffixes are removed, so different words share a stem (communication and communities both give commun).
- Derivational errors with non-word stems (happily → happili, reliable → reliabl, business → busi): the suffix is only partly removed.

## Question 6 – Error Analysis of a Finite-State Morphological Parser

### Task 1 – Logical errors

- word[:-2] always removes two characters. For cars it gives ca, and for cities it gives citi.
- boxes → box is correct only by chance (the ending is -es), so the rule is not reliable.
- Words that do not end in s but are plural (children) are labelled Singular.
- Every word that ends in s is called plural, including singular words such as class and bus.
- There is no lexicon check, so the parser cannot tell a real root from an invented one.

### Task 2 – Why it fails for different plural rules

English has several plural patterns: regular -s (cars), -es after s, x, z, ch, sh (boxes, buses, churches), consonant + y → -ies (cities), -f/-fe → -ves (wolves, knives), and irregular forms (children, mice, men). One fixed rule can only match one of them.

### Task 3 and 4 – Corrected parser and execution

```
# Q6: Rule-based plural-noun parser - original (faulty) and corrected versions

def parser_original(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"

print("ORIGINAL PARSER")
for w in ["cars", "boxes", "cities", "children"]:
    print(" ", w, "->", parser_original(w))

# Stand-in lexicon (with NLTK use: from nltk.corpus import wordnet; wordnet.synsets(root, "n"))
LEXICON = {"car", "box", "city", "child", "bus", "class", "glass", "wolf", "knife", "man",
           "woman", "mouse", "foot", "dog", "church", "wish", "boy", "lady", "person"}
IRREGULAR = {"children": "child", "men": "man", "women": "woman", "mice": "mouse",
             "feet": "foot", "people": "person"}

# (plural ending, replacement, rule name) - most specific rule first
RULES = [("ies", "y", "-ies -> -y"), ("ves", "f", "-ves -> -f"), ("ves", "fe", "-ves -> -fe"),
         ("es", "", "-es"), ("s", "", "regular -s")]

def parser(word):
    if word in IRREGULAR:                       # 1. irregular plurals
        return IRREGULAR[word], "Plural Noun (irregular)"
    if word in LEXICON:                         # 2. already a known singular
        return word, "Singular"
    for ending, repl, name in RULES:            # 3. regular rules, lexicon-checked
        if word.endswith(ending):
            root = word[:-len(ending)] + repl
            if root in LEXICON:
```

```
                return root, f"Plural Noun ({name})"
    return word, "Singular (unknown word)"

print("\nCORRECTED PARSER")
for w in ["cars", "boxes", "cities", "children", "buses", "wolves", "knives",
          "glasses", "class", "mice", "men", "churches"]:
    print(f"  {w:<9} -> {parser(w)}")
```

```
ORIGINAL PARSER
  cars -> ('ca', 'Plural Noun')
  boxes -> ('box', 'Plural Noun')
  cities -> ('citi', 'Plural Noun')
  children -> ('children', 'Singular')

CORRECTED PARSER
cars      -> ('car', 'Plural Noun (regular -s)')
boxes     -> ('box', 'Plural Noun (-es)')
cities    -> ('city', 'Plural Noun (-ies -> -y)')
children  -> ('child', 'Plural Noun (irregular)')
buses     -> ('bus', 'Plural Noun (-es)')
wolves    -> ('wolf', 'Plural Noun (-ves -> -f)')
knives    -> ('knife', 'Plural Noun (-ves -> -fe)')
glasses   -> ('glass', 'Plural Noun (-es)')
class     -> ('class', 'Singular')
```

### Output

ORIGINAL PARSER
cars -> ('ca', 'Plural Noun')
boxes -> ('box', 'Plural Noun')
cities -> ('citi', 'Plural Noun')
children -> ('children', 'Singular')

CORRECTED PARSER
cars -> ('car', 'Plural Noun (regular -s)')
boxes -> ('box', 'Plural Noun (-es)')
cities -> ('city', 'Plural Noun (-ies -> -y)')
children -> ('child', 'Plural Noun (irregular)')
buses -> ('bus', 'Plural Noun (-es)')
wolves -> ('wolf', 'Plural Noun (-ves -> -f)')
knives -> ('knife', 'Plural Noun (-ves -> -fe)')
glasses -> ('glass', 'Plural Noun (-es)')
class -> ('class', 'Singular')
mice -> ('mouse', 'Plural Noun (irregular)')
men -> ('man', 'Plural Noun (irregular)')
churches -> ('church', 'Plural Noun (-es)')

### Task 5 – Limitations of rule-based finite-state parsing

- Irregular forms must be listed one by one, and the list is never complete.

- Rules need a lexicon to check the result. Unknown words, names and new words fail.
- Some words are ambiguous: glasses, news and series can be singular or plural, and runs can be a noun or a verb. Without context or part-of-speech information the parser cannot decide.
- Every exception creates a new rule, so large rule sets become hard to maintain and may conflict.
- Spelling rules differ between languages, so a parser cannot be reused directly for another language.

**Question 7 – Error Analysis of Morphological Feature Extraction**

**Task 1 – Preprocessing error**

Stemming is applied after CountVectorizer has already built the matrix. Only the list of feature names is stemmed. The matrix X still has one column for each unstemmed word, and the stemmed list no longer matches the columns in a one-to-one way.

**Task 2 – Why this gives a redundant vocabulary**

The matrix has 9 columns, but the stemmed feature list has only 4 different stems (organ three times, run twice, studi three times, runner once). Counts of the same stem remain split over several columns, so the model treats running and runs as different features and does not gain the intended reduction in vocabulary.

**Task 3 and 4 – Corrected pipeline and execution**

The correction is to stem inside the vectorizer, by passing a custom analyzer. Tokenization, lowercasing and stemming then happen before the vocabulary is built.

```
# Q7: Stemming BEFORE vectorizing (corrected) vs AFTER vectorizing (original, faulty)
import re, time
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import cross_val_predict, LeaveOneOut
from sklearn.metrics import accuracy_score

stemmer = PorterStemmer()

# ---------- ORIGINAL (faulty) program ----------
documents = ["running runners runs", "studies studied studying", "organization organized organizer"]
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(documents)
features = [stemmer.stem(word) for word in vectorizer.get_feature_names_out()]
print("ORIGINAL: matrix columns =", X.shape[1], "| features =", features)
print("          unique features =", len(set(features)), "(duplicates are still separate columns)")

# ---------- CORRECTED program ----------
```

```
def stem_analyzer(doc):
        return [stemmer.stem(w) for w in re.findall(r"[a-z]+", doc.lower())]

vec2 = CountVectorizer(analyzer=stem_analyzer)
X2 = vec2.fit_transform(documents)
print("\nCORRECTED: matrix columns =", X2.shape[1], "| features =",
list(vec2.get_feature_names_out()))

# ---------- Comparison on a small labelled sample (use 20 Newsgroups for the full run) ----------
docs = [
        "runners were running along the track while the coach runs drills",
        "she runs daily and the runner trained for racing",
        "running clubs organized races and the runners trained hard",
        "the coach trains runners and races are run on weekends",
        "training schedules help runners improve their running times",
        "he trained hard and ran the race with other runners",
        "students were studying for exams while the teacher studies lessons",
        "she studies nightly and the student studied for testing",
        "study groups organized lessons and the students learned quickly",
        "the teacher teaches students and lessons are studied on weekends",
        "learning schedules help students improve their studying habits",
        "he studied hard and learned the lesson with other students",
]
y = [0] * 6 + [1] * 6

def evaluate(vec, repeats=200):
        start = time.perf_counter()
        for _ in range(repeats):
                Xv = vec.fit_transform(docs)
        ms = (time.perf_counter() - start) / repeats * 1000
        pred = cross_val_predict(MultinomialNB(), Xv, y, cv=LeaveOneOut())
        return Xv.shape[1], accuracy_score(y, pred), ms

print("\nCOMPARISON (12 labelled sample documents, leave-one-out Naive Bayes)")
for name, vec in [("Original (no stemming in matrix)", CountVectorizer(analyzer=lambda d:
re.findall(r"[a-z]+", d.lower()))),
                                ("Corrected (stem inside analyzer)",
                                CountVectorizer(analyzer=stem_analyzer))]:
        v, acc, ms = evaluate(vec)
        print(f"  {name:<33} vocabulary = {v:<3} accuracy = {acc:.3f}  time = {ms:.2f} ms")
```

```
Run started
Initializing environment
Installing packages
Running code
ORIGINAL: matrix columns = 9 | features = ['organ', 'organ', 'organ', 'runner', 'run', 'run', 'studi',
'studi', 'studi']
unique features = 4 (duplicates are still separate columns)

CORRECTED: matrix columns =
features = ['organ', 'run', 'runner', 'studi']

COMPARISON
Original (no stemming in matrix)  vocabulary = 55  accuracy = 1.000 time = 0.59 ms
Corrected (stem inside analyzer)  vocabulary = 42  accuracy = 1.000 time = 2.60 ms
Run completed in 7,559.300ms
```

### Output

ORIGINAL: matrix columns = 9 | features = ['organ', 'organ', 'organ', 'runner', 'run', 'run', 'studi', 'studi', 'studi']
unique features = 4 (duplicates are still separate columns)

CORRECTED: matrix columns = 4 | features = ['organ', 'run', 'runner', 'studi']

COMPARISON (12 labelled sample documents, leave-one-out Naive Bayes)
Original (no stemming in matrix) vocabulary = 55 accuracy = 1.000 time = 0.30 ms
Corrected (stem inside analyzer) vocabulary = 42 accuracy = 1.000 time = 1.19 ms

### Task 5 – Comparison (12-document sample)

| Measure | Before correction | After correction | Change |
|---|---|---|---|
| Vocabulary size | 55 | 42 | 13 fewer features |
| Classification accuracy | 1.000 | 1.000 | No difference on this sample |
| Processing time (approx.) | 0.3 ms | 1.2 ms | Slower, because each word is stemmed |

Accuracy is the same on this small sample, so no accuracy gain can be claimed from it. On a large corpus such as 20 Newsgroups, the effect of a smaller vocabulary on accuracy and memory should be measured again. The timing values vary slightly from run to run.

### Task 6 – How the corrected pipeline helps

- Counts of word forms with the same stem are added into one feature, so the model sees stronger evidence for each concept.

- The matrix is narrower, so memory use and training cost are lower and the risk of over-fitting to rare word forms is reduced.
- The stems are built before the vocabulary, so the features match the matrix columns exactly.
- The extra stemming time can be reduced by caching the stem of each word.